🕯️

First-principles precalciner model: MOK prototype

Status	Not discussed
Plants	Heidelberg Mokra
Created	February 15, 2026 9:08 AM
Last edited	April 23, 2026 8:44 AM
Features	
Author	Michael Craig
Tags	
Ticket	https://linear.app/carbonre/issue/ML-2860/document-review-and-tidy-up-heat-transfer-model
Project	Physics-based models

Motivation

In 1929, hailing the success of quantum mechanics, Paul Dirac wrote:

> *"The underlying physical laws necessary for the mathematical theory of a large part of physics and the whole of chemistry are thus completely known, and the difficulty is only that the exact application of these laws leads to equations much too complicated to be soluble."*

He was basically saying the Schrodinger equation was a total micdrop, slay. Once you have the right equation governing a system, you have solved the research

problem. What remains is computation: finding approximate methods good enough to be useful

We need to derive this governing equation in cement in order to define optimal controller behaviour. With CC, we can be more ambitious about the scientific problems we tackle, since these tools compress work that would've taken many weeks to a few days of focused work for one researcher

Given such a governing equation, we can use it as a foundation to:

- Understand power flows in cement plants more clearly. When our governing equation cannot represent our data, we can ask *which part is wrong* and *why.* This interpretability makes model errors easier to audit and debug than black box models
- Train policies using a physics model
- Train a data-driven model to predict the residuals on top of the mechanistic model

Method

Specc'd out a plan with cc using an old write up Heat simulator, as inspiration, and knowing that I wanted to start by using shutdown and startup periods as boundary conditions to define start and end points for the simulations from which we could progressively add complexity

Then iteratively fit across shutdown and startup events in mokra, since they often have brief, planned stoppages

We have `HeatFlowContribution` class which is the object for handling changes in each of the temperatures to consider, which at the moment are the 3 temperatures closest to the bottom in the pc

```
class HeatFlowContribution(BaseModel):
    """Heat flow contribution to each stage.

    Attributes:
        dT_pc: Temperature change rate for PC (C/min)
        dT_cy4: Temperature change rate for CY4 (C/min)
```

```
        dT_cy3: Temperature change rate for CY3 (C/min)
        q_total: Total energy flow for tracking (GJ/h)
    """

    dT_pc: float = 0.0
    dT_cy4: float = 0.0
    dT_cy3: float = 0.0
    q_total: float = 0.0

    def __add__(self, other: HeatFlowContribution) -> HeatFlo
wContribution:
        return HeatFlowContribution(
            dT_pc=self.dT_pc + other.dT_pc,
            dT_cy4=self.dT_cy4 + other.dT_cy4,
            dT_cy3=self.dT_cy3 + other.dT_cy3,
            q_total=self.q_total + other.q_total,
        )

    def __neg__(self) -> HeatFlowContribution:
        return HeatFlowContribution(
            dT_pc=-self.dT_pc,
            dT_cy4=-self.dT_cy4,
            dT_cy3=-self.dT_cy3,
            q_total=-self.q_total,
        )
```

We define ODEs which describe how heat is distributed in different stages of the preheater

We then define a forward Euler scheme to step these ODE heat flows in 5 min timesteps whereby each ‘module’ represents a distinct heat transfer concept

```
    # Compute contributions from all modules
    total_dT = HeatFlowContribution()
    for module in self.modules:
```

```
            contribution = module.compute(state, inputs)
            total_dT = total_dT + contribution
            module_contributions[module.name][i] = contribution.q
_total
```

Where there is a free parameter in the ODE we need to fit using past events

This was done by minimising error over the whole event. We could potentially instead treat every timestep as an event and minimise that error subject to all the inputs

Now we describe the different modules

Ambient loss

During a stoppage, the plant is coming into equilibrium with the environment, so the physics is simple, the heat should transfer from hot → cold, tending towards ambient temperature roughly according to newtons law of cooling

This term looks like

$$\frac{dT_i}{dt} = k(T_{\text{ambient}} - T)$$

the ambient temperature is changing in absolute value (reducing) and likely moving position, but to get around that we have a time varying k

▼ code

```
class AmbientLossModule:
    """Conduction/radiation heat loss to environment."""

    name = "ambient_loss"

    def __init__(self, params: AmbientLossParams | None =
None, enabled: bool = True):
        self.params = params or AmbientLossParams()
        self.enabled = enabled

    def compute(
```

```
        self,
        state: SimulationState,
        inputs: SimulationInputs,
    ) -> HeatFlowContribution:
        if not self.enabled:
            return HeatFlowContribution()

        p = self.params

        # Compute time-varying k_ambient based on phase
        if state.in_warmup_phase and state.t_since_fuel_re
start is not None:
            decay = np.exp(-state.t_since_fuel_restart /
p.tau_k_warmup)
            k_ambient = p.k_ambient_warmup_late + (p.k_amb
ient_warmup_early - p.k_ambient_warmup_late) * decay
        else:
            decay = np.exp(-state.t_mins / p.tau_k_transit
ion)
            k_ambient = p.k_ambient_late + (p.k_ambient_ea
rly - p.k_ambient_late) * decay

        # Heat loss proportional to temperature difference
        loss_pc = k_ambient * (state.temps.t_pc - p.t_ambi
ent)
        loss_cy4 = k_ambient * (state.temps.t_cy4 - p.t_am
bient)
        loss_cy3 = k_ambient * (state.temps.t_cy3 - p.t_am
bient)

        return HeatFlowContribution(
            dT_pc=-loss_pc,
            dT_cy4=-loss_cy4,
            dT_cy3=-loss_cy3,
```

```
            q_total=0.0,  # Could compute if needed
        )
```

Gas flow heat exchange

Heat flows from hot to cold, i.e. from the PC → CY4, CY4 → CY3

▼ code

```
class GasFlowExchangeModule:
    """Inter-stage heat exchange via upward gas flow.

    Hot gas flows kiln → PC → CY4 → CY3. Each stage heats
the one above
    proportional to the temperature difference. The source
stage cools
    by the same amount (energy conservation).
    """

    name = "gas_flow_exchange"

    def __init__(
        self, params: GasFlowExchangeParams | None = None,
enabled: bool = True
    ):
        self.params = params or GasFlowExchangeParams()
        self.enabled = enabled

    def compute(
        self,
        state: SimulationState,
        inputs: SimulationInputs,
    ) -> HeatFlowContribution:
        if not self.enabled:
            return HeatFlowContribution()
```

```
        p = self.params
        temps = state.temps

        # Kiln gas → PC (kiln_temp is an external input, n
o cooling term for kiln)
        dT_kiln_to_pc = p.k_kiln_to_pc * max(0, inputs.kil
n_temp - temps.t_pc)

        # PC gas → CY4 (PC cools, CY4 heats)
        delta_pc_cy4 = p.k_pc_to_cy4 * max(0, temps.t_pc -
temps.t_cy4)

        # CY4 gas → CY3 (CY4 cools, CY3 heats)
        delta_cy4_cy3 = p.k_cy4_to_cy3 * max(0, temps.t_cy
4 - temps.t_cy3)

        return HeatFlowContribution(
            dT_pc=dT_kiln_to_pc - delta_pc_cy4,
            dT_cy4=delta_pc_cy4 - delta_cy4_cy3,
            dT_cy3=delta_cy4_cy3,
        )
```

This will need extension if we consider 3air temp

Coupling to fuel

Heat is input as fuel in the kiln during restarts, and this heat is transferred via gases to the precalciner. Really the temperature change occurs because of kiln temperature but for now since I don't trust pyrometer temperatures to be well-behaved during startups have lumped the heat from that temperature as ~ the heat from the kiln fuels

▼ Therefore the coupling to fuel is as below:

```
class FuelHeatingModule:
    """Heat from fuel combustion in PC and kiln — applied
to PC only.
```

```
    CY4/CY3 receive heat indirectly via GasFlowExchangeMod
ule.
    """

    name = "fuel_heating"

    def __init__(self, params: FuelHeatingParams | None =
None, enabled: bool = True):
        self.params = params or FuelHeatingParams()
        self.enabled = enabled

    def compute(
        self,
        state: SimulationState,
        inputs: SimulationInputs,
    ) -> HeatFlowContribution:
        if not self.enabled:
            return HeatFlowContribution()

        p = self.params

        # Linear fuel model, capped
        q_pc = min(max(0, inputs.pc_fuel_power) * p.pc_fue
l_slope, p.pc_fuel_max)
        q_kiln = min(max(0, inputs.kiln_fuel_power) * p.ki
ln_fuel_slope, p.kiln_fuel_max)
        q_total = q_pc + q_kiln

        if q_total <= 0:
            return HeatFlowContribution()

        # All fuel heat goes to PC (GJ/h → °C/h)
        return HeatFlowContribution(
            dT_pc=q_total / p.thermal_mass_effective,
```

```
            q_total=q_total,
        )
```

As an extension to this, we would more carefully track the difference between the source of energy input being the precalciner, and the source of energy to the precalciner being either the back of the kiln or the 3air

Mass flow heat sink

When we start to put material through the system, that heat is taken up in the pre-calciner since the temperature in the silo is at about 90 degrees

$$\frac{dT_{i,g}}{dt} = \dot{m}_i c_{p,\text{solid}} (T_{i,g} - T_{\text{i,m,solid}})$$

The heat that is sunk at each stage varies as a function of the total mass flow at that stage $\dot{m}_i$ since there is more/less mass flow at a given point because calcination moves some solid CaCO3 to gaseous CO2, but the mass fraction we muliply is `s_ph_sil_tput`. For now we have set that fraction to be fixed and decreasing across stages

What is interesting about this term is that it contains a term that is dependent on the composition of the input material, the specific heat capacity of the solids! So we could begin to start actually using LSF instead of chanting at a linear regressor to use it

This is not a complete picture, since some particulate matter is recycled to heat the rest of the plant, and $\dot{m}_i$ is a function of temperature and partial pressures

▼ code, good bit of complexity could be reduced by making assumptions or seeing the wood for the trees a bit probably

```
class MaterialHeatSinkModule:
    """Sensible heat absorbed by raw meal flow."""

    name = "material_heat_sink"

    def __init__(self, params: MaterialHeatSinkParams | No
ne = None, enabled: bool = True):
```

```
        self.params = params or MaterialHeatSinkParams()
        self.enabled = enabled

    def _estimate_material_temperatures(
        self, temps: StageTemperatures
    ) -> tuple[float, float, float]:
        """Estimate material temperature at each stage inl
et."""
        p = self.params

        # Material enters CY3 at inlet temperature
        t_mat_cy3 = p.t_material_inlet

        # Material heats up as it descends
        t_mat_exit_cy3 = t_mat_cy3 + p.heat_transfer_effic
iency * (temps.t_cy3 - t_mat_cy3)
        t_mat_cy4 = t_mat_exit_cy3

        t_mat_exit_cy4 = t_mat_cy4 + p.heat_transfer_effic
iency * (temps.t_cy4 - t_mat_cy4)
        t_mat_pc = t_mat_exit_cy4

        return t_mat_cy3, t_mat_cy4, t_mat_pc

    def _compute_heat_sink_at_stage(
        self,
        t_stage: float,
        t_material_inlet: float,
        m_dot_material: float,
        mass_fraction: float,
    ) -> float:
        """Compute heat absorbed at a single stage (GJ/
h)."""
        p = self.params

        if m_dot_material <= 0 or t_stage <= t_material_in
```

```
let:
            return 0.0

        # Mass flow at this stage (t/h -> kg/h)
        m_dot_stage = m_dot_material * mass_fraction * 100
0

        # Q = m_dot * c_p * deltaT (kJ/h -> GJ/h)
        return m_dot_stage * p.c_p_material * (t_stage - t
_material_inlet) / 1e6

    def compute(
        self,
        state: SimulationState,
        inputs: SimulationInputs,
    ) -> HeatFlowContribution:
        if not self.enabled:
            return HeatFlowContribution()

        p = self.params

        if inputs.material_flow < p.material_flow_threshol
d:
            return HeatFlowContribution()

        # Estimate material temperatures
        t_mat_cy3, t_mat_cy4, t_mat_pc = self._estimate_ma
terial_temperatures(state.temps)

        # Heat absorbed at each stage
        q_mat_cy3 = self._compute_heat_sink_at_stage(
            state.temps.t_cy3, t_mat_cy3, inputs.material_
flow, p.mass_frac_cy3
        )
        q_mat_cy4 = self._compute_heat_sink_at_stage(
            state.temps.t_cy4, t_mat_cy4, inputs.material_
```

```
flow, p.mass_frac_cy4
        )
        q_mat_pc = self._compute_heat_sink_at_stage(
            state.temps.t_pc, t_mat_pc, inputs.material_fl
ow, p.mass_frac_pc
        )

        q_mat_total = (q_mat_cy3 + q_mat_cy4 + q_mat_pc) *
p.material_cooling_scale

        # Convert to temperature change (GJ/h → °C/h)
        dT_mat_pc = q_mat_pc * p.material_cooling_scale /
p.thermal_mass_effective
        dT_mat_cy4 = q_mat_cy4 * p.material_cooling_scale
/ p.thermal_mass_effective
        dT_mat_cy3 = q_mat_cy3 * p.material_cooling_scale
/ p.thermal_mass_effective

        return HeatFlowContribution(
            dT_pc=-dT_mat_pc,
            dT_cy4=-dT_mat_cy4,
            dT_cy3=-dT_mat_cy3,
            q_total=q_mat_total,
        )
```

An extension of this, is that this module should talk to the calcination module

Calcination heat sink

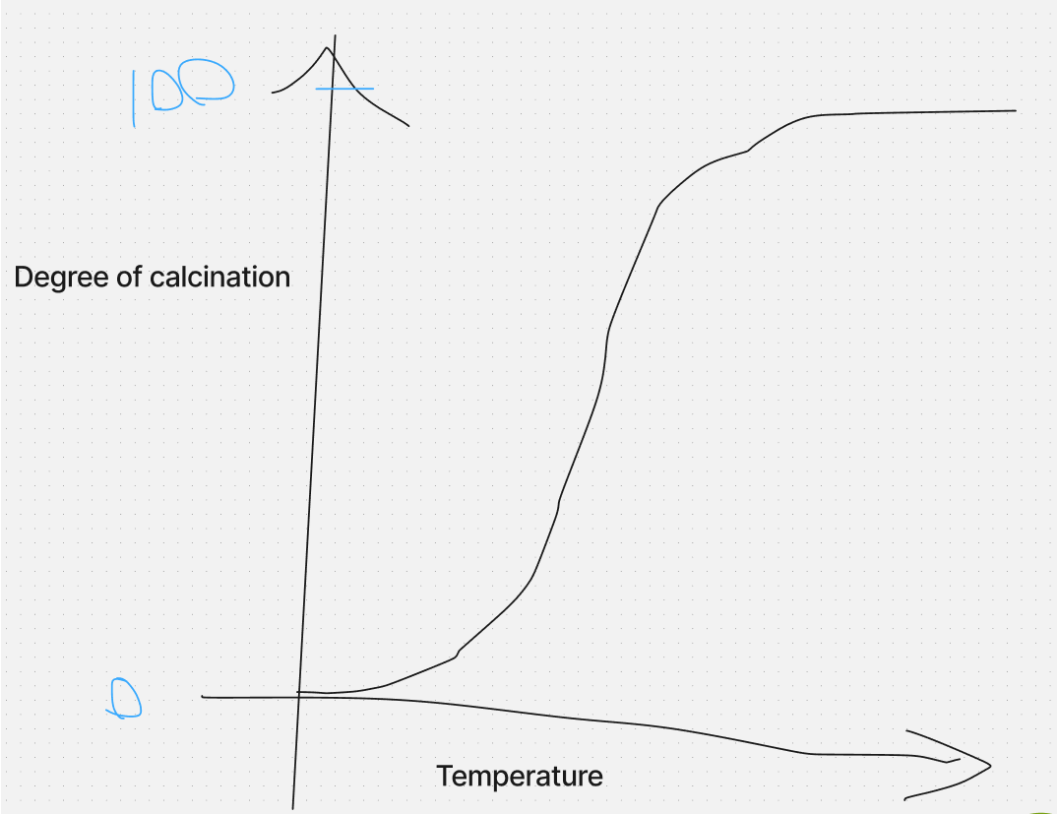
Heat is also sank into calcination to drive:

$$\mathrm{CaCO_3} \rightarrow \mathrm{CaO} + \mathrm{CO_2} - 1.78\mathrm{MJ/kg}$$

We need to pay the piper here to make cement! The rest is managing the painful fact we have that annoying penalty on the right hand side

This is in proportion to the amount of material already calcined which makes it tricky to think through how to actually set the mass fractions to calcine, but just started with a temperature dependent fraction and a fraction of calcination assumed per stage

The amount of calcination occurring is a function of the temperature, here the onset temp was 700 C and full calcination temp 850



simplified sigmoid for doc temp

▼ code

```
class CalcinationHeatSinkModule:
    """Endothermic calcination reaction heat sink."""

    name = "calcination_heat_sink"

    def __init__(self, params: CalcinationHeatSinkParams |
None = None, enabled: bool = True):
```

```
        self.params = params or CalcinationHeatSinkParams
()
        self.enabled = enabled

    def _compute_calcination_at_stage(
        self,
        m_dot_material: float,
        t_stage: float,
        calcination_fraction_stage: float,
    ) -> float:
        """Compute calcination heat sink at a single stage
(GJ/h)."""
        p = self.params

        if m_dot_material <= 0 or calcination_fraction_sta
ge <= 0:
            return 0.0

        # Temperature-dependent calcination rate
        if t_stage < p.t_calcination_onset:
            temp_factor = 0.0
        elif t_stage >= p.t_calcination_full:
            temp_factor = 1.0
        else:
            # this should maybe be a sigmoid?
            temp_factor = (t_stage - p.t_calcination_onse
t) / (
                p.t_calcination_full - p.t_calcination_ons
et
            )

        # Mass of CaCO3 in raw meal (kg/h)
        m_calcite_total = m_dot_material * 1000 * p.limest
one_fraction

        # Mass calcined at this stage
```

```
        m_calcite_stage = m_calcite_total * calcination_fr
action_stage * temp_factor

        # Energy absorbed (MJ/h -> GJ/h)
        return m_calcite_stage * p.calcination_enthalpy /
1000
```

Air flow heat sink

$$\frac{dT_{i,g}}{dt} = \dot{m}_{i,\text{air}} c_{p,\text{air}} \rho_i (T_{i,g} - T_{i,\text{air}})$$

where ρ_i is a fixed, per-stage fraction of the heat loss due to air at that stage.

This will need to be revisited if we extend to representing temperatures other than pc/cy4/cy3 e.g. the cooler temperature, which is very sensitive to air flow changes

We approximate the mass flow of air using the fan speed as a rough proxy for it

▼ code

```
class AirCoolingModule:
    """Convective cooling from fan-driven air flow."""

    name = "air_cooling"

    def __init__(self, params: AirCoolingParams | None = N
one, enabled: bool = True):
        self.params = params or AirCoolingParams()
        self.enabled = enabled

    def _compute_air_mass_flow(self, fan_speed: float) ->
float:
        """Estimate air mass flow from fan speed (kg/
h)."""
        if fan_speed <= 0:
            return 0.0
```

```
        p = self.params
        volumetric_flow = fan_speed * p.fan_flow_coefficie
nt
        return volumetric_flow * p.gas_density

    def compute(
        self,
        state: SimulationState,
        inputs: SimulationInputs,
    ) -> HeatFlowContribution:
        if not self.enabled:
            return HeatFlowContribution()

        p = self.params
        m_dot_air = self._compute_air_mass_flow(inputs.fan
_speed)

        if m_dot_air <= 0:
            return HeatFlowContribution()

        # Average stage temperature
        t_avg = (state.temps.t_pc + state.temps.t_cy4 + st
ate.temps.t_cy3) / 3

        if t_avg <= p.t_ambient:
            return HeatFlowContribution()

        # Q = m_dot * c_p * deltaT (kJ/h -> GJ/h)
        q_air_total = m_dot_air * p.c_p_air * (t_avg - p.t
_ambient) / 1e6
        q_air_total *= p.air_cooling_scale

        # Distribute across stages (GJ/h → °C/h)
        dT_air_pc = q_air_total * p.air_loss_frac_pc / p.t
hermal_mass_effective
        dT_air_cy4 = q_air_total * p.air_loss_frac_cy4 /
```

```
p.thermal_mass_effective
        dT_air_cy3 = q_air_total * p.air_loss_frac_cy3 /
p.thermal_mass_effective

        return HeatFlowContribution(
            dT_pc=-dT_air_pc,
            dT_cy4=-dT_air_cy4,
            dT_cy3=-dT_air_cy3,
            q_total=q_air_total,
        )
```

Results

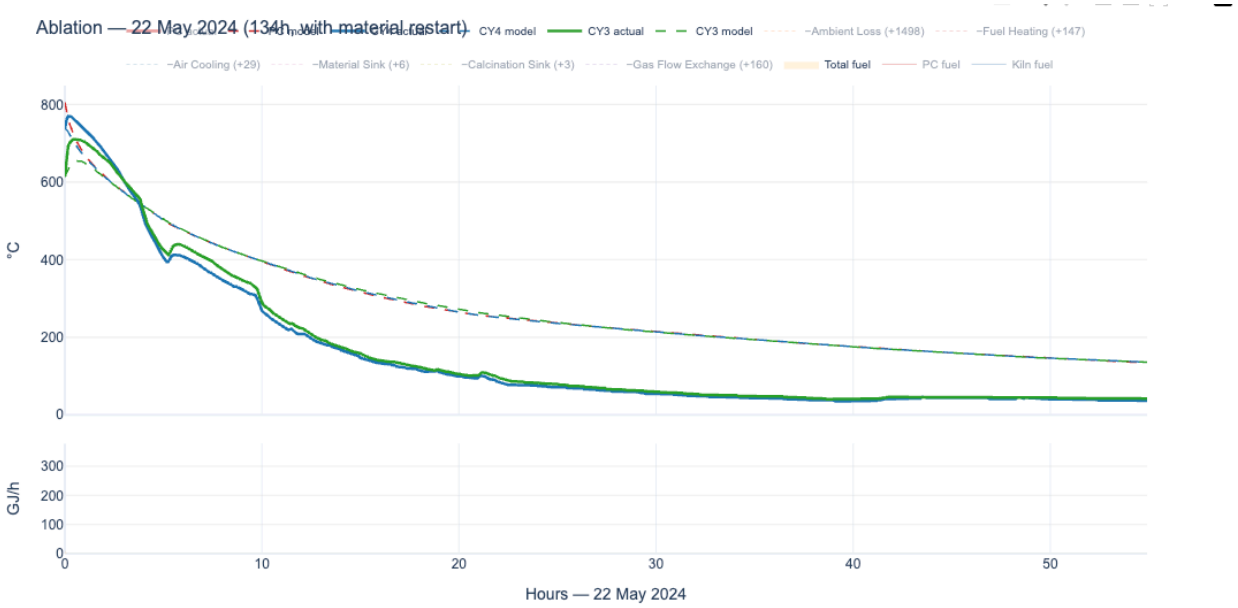
Here the goal was to capture the dynamics and show proof-of-life for first principles modelling

To do this, ran experiments where the temperatures were evolved subject to the equations above from time t = shutdown, which was determined by checking the fuels, and then run for as many hours as it took to see the plant back up-and-running again while replaying the fuels

Timesteps were in steps of 5min

Cooling down

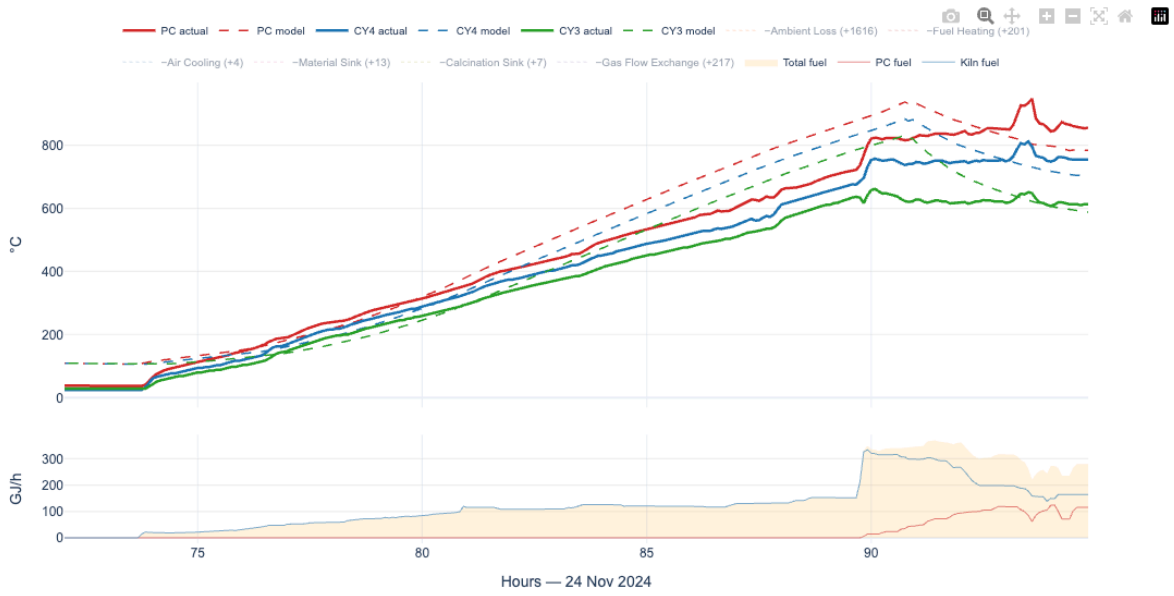
Broadly, we can capture the dynamics of shutdowns reasonably well, but tend to have a positive bias (we think temperatures are too high). I think this is because we are not considering losses to ambient due to the cooler, or we have fitted the thermal mass parameter incorrectly by starting the simulations at time when the temperatures are increasing because of latent heat (i.e. there's heat energy → into the system we are not considering when we set the simulation to start at dE=0, where dE=0 is when the fuel feeding rate is 0



Warming up

These periods are more interesting, since we only run when fuel is being burnt, and because more things are happening

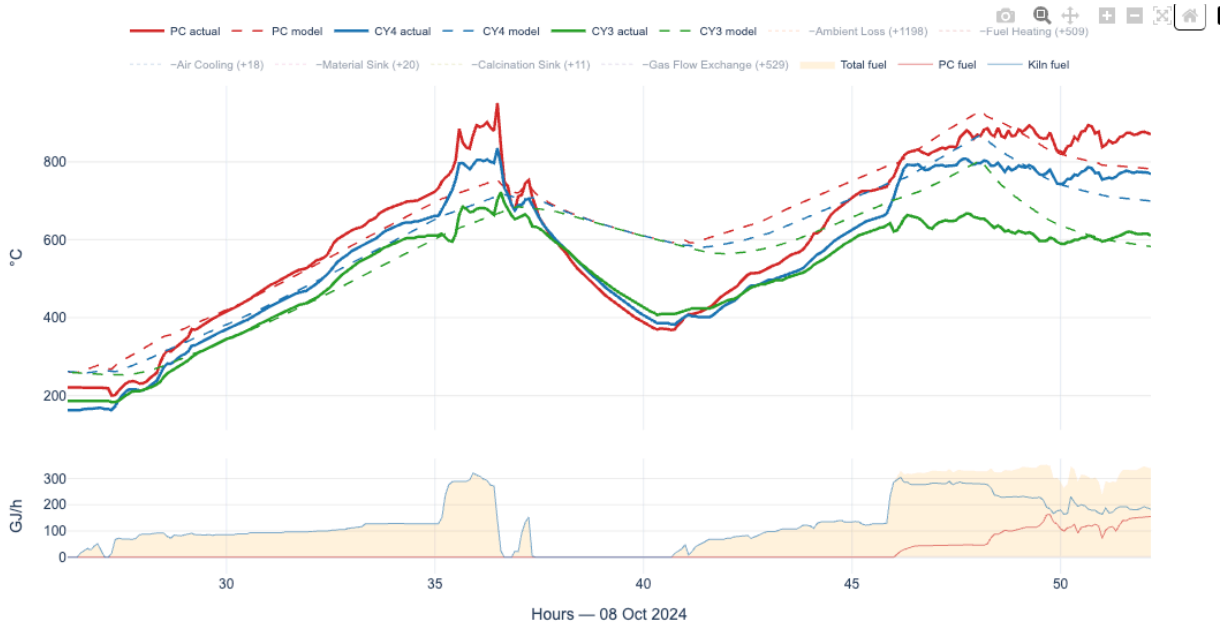
The broad picture is that we can capture the dynamics and each of the modules helps to differing degrees



We don’t seem to capture the heat dynamics of heat flowing to material very well, and I think there is something to that about the fact that we don’t consider that the heat is flowing from the entrained solids

In addition, I think the calcination model means the temperatures will settle to an equilibrium point defined by the pre-set temperature of 0/full calcination, when really we need to have a model for the presumed calcination at a given temperature and point in the plant

We could also simply fit this at any given time, which is a potential way to arrive at a degree of calcination if we asked “what DoC would explain this temperature profile”. That would assume everything else about your model is correct which is shaky



Did an ablation of each of the modules to determine which parts are good/bad or could be simplified this is helpful in preventing claude from going insane trying to hunt for MAE minimising while overcomplicating things

Summary from cc

```
  5 events evaluated (27–134h duration, all with material res
tart).
```

```
  Module Importance (mean MAE delta when removed)

  Module: Ambient Loss
  Mean Δ MAE (°C): +1326
  Verdict: By far the most critical — model collapses without
it
  ──────────────────────────────────────────
  Module: Gas Flow Exchange
  Mean Δ MAE (°C): +339
  Verdict: Very important, especially for PC stage
  ──────────────────────────────────────────
  Module: Fuel Heating
  Mean Δ MAE (°C): +268
  Verdict: Important, though slightly hurts on the Aug event
(−61)
  ──────────────────────────────────────────
  Module: Air Cooling
  Mean Δ MAE (°C): +23
  Verdict: Modest help, most impact on longer/hotter events
  ──────────────────────────────────────────
  Module: Material Sink
  Mean Δ MAE (°C): +16
  Verdict: Small but consistent improvement
  ──────────────────────────────────────────
  Module: Calcination Sink
  Mean Δ MAE (°C): +8
  Verdict: Smallest contributor, but never hurts

  Key observations

  - All modules help on average — no module should be remove
d.
  - Ambient Loss dominates — it's the backbone of the model
(removing it adds
  ~1300°C total MAE).
```

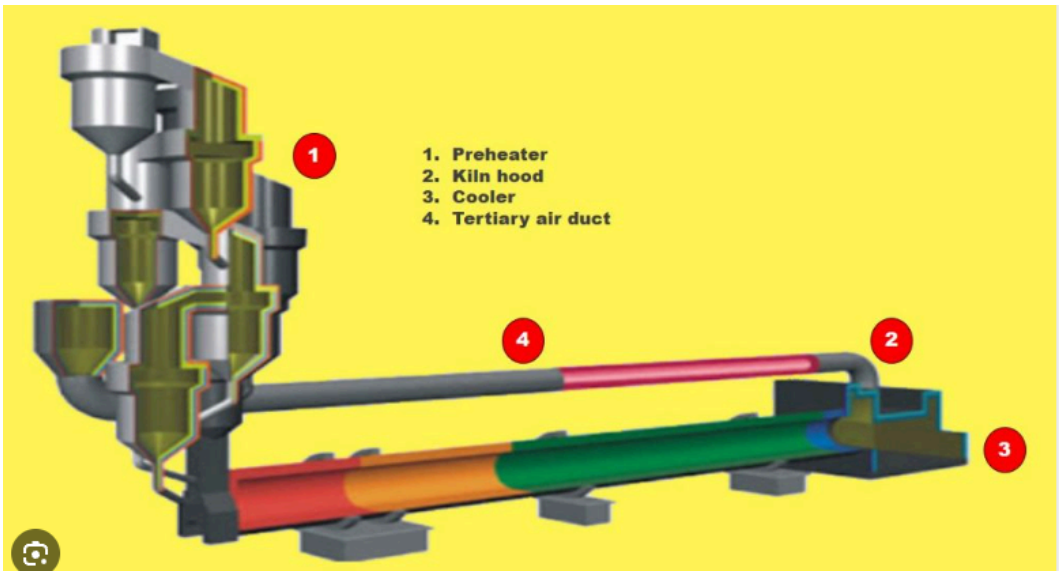
```
  - Gas Flow Exchange and Fuel Heating are the next tier, eac
h adding 250–340°C
  MAE when removed.
  - Fuel Heating on Aug 6 event is the only case where removi
ng a module helped
  (−61°C). That event has unusually high mean fuel (337 GJ/h)
— the fuel
  response may be overshooting there.
  - Material/Calcination sinks have small but positive deltas
— they matter most
   during the material restart periods (e.g., May 20 event sh
ows +28 for
  material sink).
  - Baseline full-model MAE ranges from 126°C (short event) t
o 650°C (66h event
  with high fuel), suggesting the model struggles more with l
onger, higher-fuel
  events.
```

ablation_results.html

It is interesting how important ambient losses are but it makes sense, we are operating so far from ambient

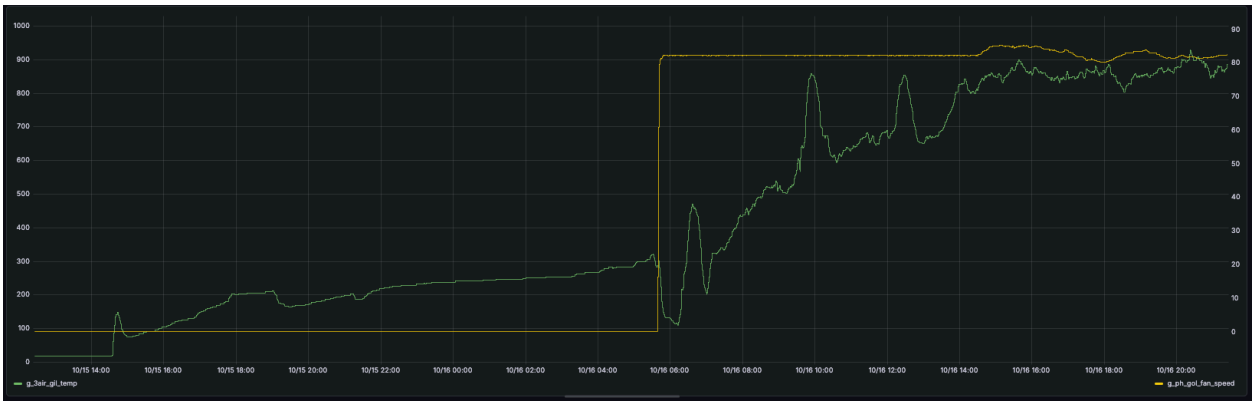
Tertiary air temperature as motivation for model extension

As the model currently exists, we do not treat the heat from the tertiary air as a sink or source of heat for the pre-heater, we should!



To capture this correctly, though, we will need to consider the cooler kiln coupling. In thinking about this, began to notice interesting things by delving into startup periods and checking how the flow of air, mass and energy affect this 3air temperature

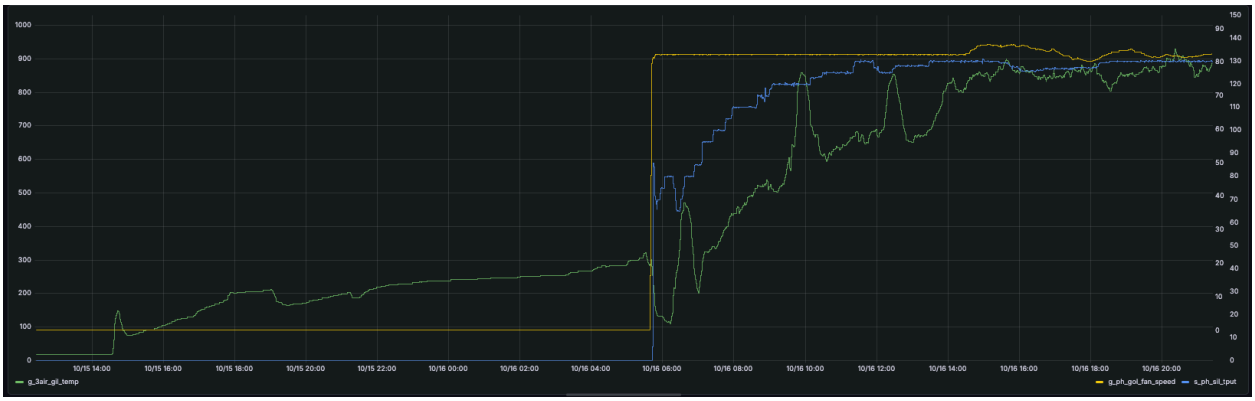
When air begins to be pulled through the system by the ID fan `g_ph_gol_fan_speed`, air at ambient temperature is sucked from the cooler into the tertiary air system, which we can see due to a concomitant drop in the 3air temperature



dynamics of tertiary air temperature at heat up
Green = Tertiary air temperature
Yellow = Fan speed

this interesting dynamic cannot be captured using the model as-is, since the 3air is not treated here in `HeatFlowContribution`

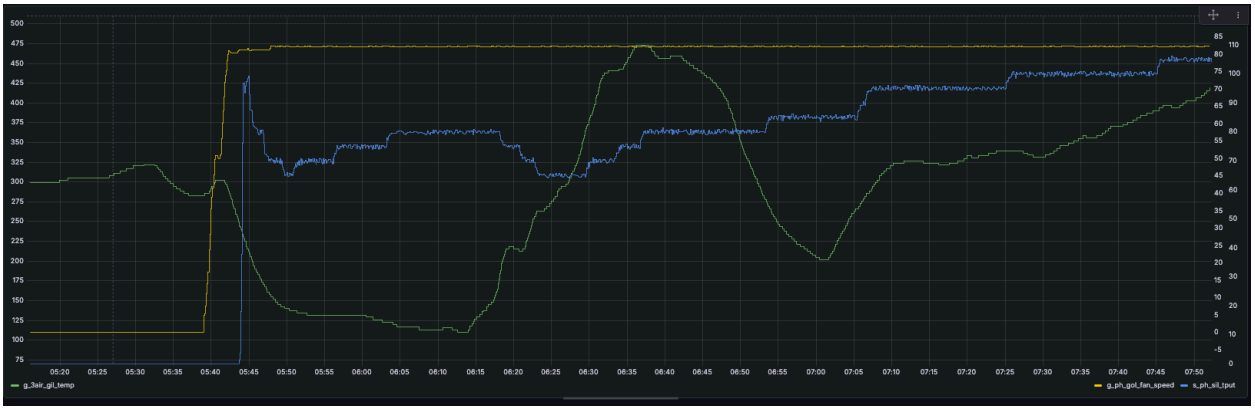
heating *up* 3air is tied to the flow of material through to the cooler



dynamics of tertiary air temperature at heat up with fan and kiln feed
Green = Tertiary air temperature
Yellow = Fan speed
Blue = kiln feed rate

I claim you can resolve the residence time of material flowing from top tower → cooler by analysing the 3air↔kiln feed dynamics

From the figure below, peaks and troughs of kiln feed lead increases and decreases to 3air about 30 minutes afterwards, although inertia is confusing things here (this assumes the heat from the material is only large enough if the kiln feed is a very specific crossover value of ~50 tons per hour)



We need to incorporate the cooler temperatures and cooler fans in order to properly capture the tertiary air, which we should try to do next in order to fully couple precalciner, kiln and cooler using a first principles model

In all, the feedback from the material flow needs to be fed into this model for it to seem truly representative and interesting.

Attempt at steady state

Pointed claude at forecasting over a much smaller horizon, using smaller timesteps having refitted the parameters but no dice, could not meaningfully beat persistence

This surprises me, I want to understand it better since it feels like we should be able to do as good as we did here

Direct forecasting while replaying control features

although one glaring difference here is that we're providing only instantaneous values as opposed to the instantaneous and lagged values as in ML models

Ultimately ended up with a model which persisted the last value of temperature, which is a pretty good approximation to what we do in the other plants 😅

Next steps

- Building in other the pieces of the plant that this modelling approach can represent, basically temperatures in other stages
- Add temperatures of the entrained solids to the sources of heat somehow, potentially as part of the work we do when we try to add tertiary air temperature
- Investigate Neural ODEs, we've effectively defined a fixed form ODE, what if we threw a Neural ODE at reproducing the temperatures in the foundation model project, or on the residual of the mechanistic temperatures
  https://arxiv.org/abs/1806.07366
  These hold promise since they'd deal with irregular timestamps and are just a quite well established tool now to do science with ML
  https://docs.kidger.site/diffrax/ (UPDATE: chatted with Patrick and he said not to use neural ODEs here!)
- Try to get use out of this in steady state for controllers
- Build a map of heat flows to inspect what it would look like to maximise exergy/efficiency as derived from these equations